

Session 5: Polymorphism Topics: Compile-time (overloading) vs Runtime (overriding). Example: uploadContent() works differently for YouTube vs Instagram.

Tasks

- 1) Create a cpp class called PaymentProcessor with two overloaded methods processPayment(): one that takes only an amount, and one that takes amount and a coupon code. Print which version is called and the final amount in each case.

Code-

```
#include<iostream>
#include <string>
using namespace std;

class PaymentProcessor{

public:
    void processPayment(double amount){
        cout<<"processing payment without coupon."<<endl;
        cout<<"finale amount: Rs"<<amount<<endl;
    }

    void processPayment(double amount,string coupon){
        double finalAmount = amount - (amount * 0.10);

        cout<<"processing payment with coupon."<<endl;
        cout<<"finale amount after discount: Rs"<<finalAmount<<endl;
    }

};

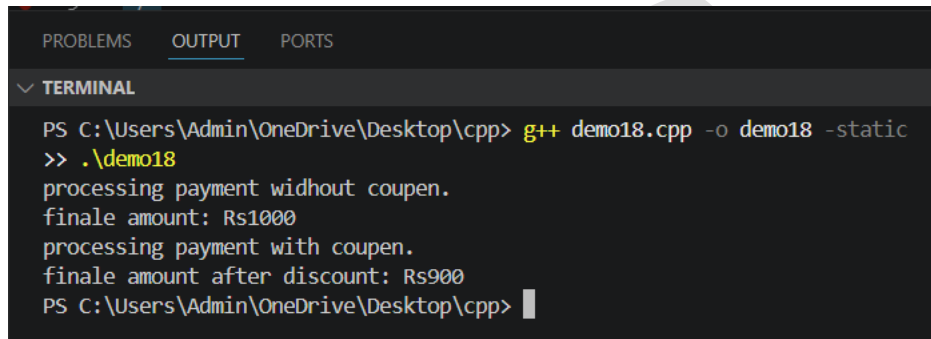
int main(){
    PaymentProcessor p1;
```

```
p1.processPayment(1000);

p1.processPayment(1000, "SAVE10");


return 0;
}
```

Output:



```
PROBLEMS OUTPUT PORTS
✓ TERMINAL
PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo18.cpp -o demo18 -static
>> .\demo18
processing payment without coupon.
finale amount: Rs1000
processing payment with coupon.
finale amount after discount: Rs900
PS C:\Users\Admin\OneDrive\Desktop\cpp>
```

- 2) Build two classes, InstagramUploader and YouTubeUploader, each with a method uploadContent(). Both should extend a base class SocialMediaUploader and override uploadContent() to print a message showing how uploading works differently for Instagram and YouTube.

Code-

```
#include<iostream>
using namespace std;

class SocialMediaUploader{
    private:

    virtual void uploadContent(){
        cout<<" we uplodated content social media every day:"<<endl;

    }

};

class InstagramUploader:public SocialMediaUploader{
    public:
    void uploadContent()override{
        cout<<"we uplodated photos and reel natrual beauty prodects on insta:"<<endl;

    }

};

class YouTubeUploader:public SocialMediaUploader{
    public:
    void uploadContent()override{
        cout<<"we uplodated the  game title,GTA ,videos in eavry weak:"<<endl;

    }

};

int main(){
    InstagramUploader i1;
    YouTubeUploader yt1;
```

```
    cout<<"instagram uplodated"<<endl;
    i1.uploadContent();
    cout<<"\nyoutube updated"<<endl;
    yt1.uploadContent();

    return 0;

}
```

Output:

```
PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo19.cpp -o demo19 -static
>> .\demo19
instagram uplodated
we uplodated photos and reel natrual beauty prodicts on insta:

youtube updated
we uplodated the game title,GTA ,videos in eavry weak:
PS C:\Users\Admin\OneDrive\Desktop\cpp>
```

3) Write a function in c++ that simulates a Flipkart-style search: overload a method searchProduct() to allow searching by product name or by product name and category. Demonstrate both usages with sample data.

Code-

```
#include<iostream>

using namespace std;

class producsearch {

public:

void searchProduct(string productname){
    cout<<"searching product name    : "<<productname<<endl;
}

void sreachProduct(string productname, string catagory){
    cout<<"searching product name    : "<<productname<<endl;
    cout<<"searching product category  : "<<catagory<<endl;
}

};

int main(){

    producsearch search;

    cout<<"\n----sreach 1----"<<endl;

    search.searchProduct("I phpne15");
```

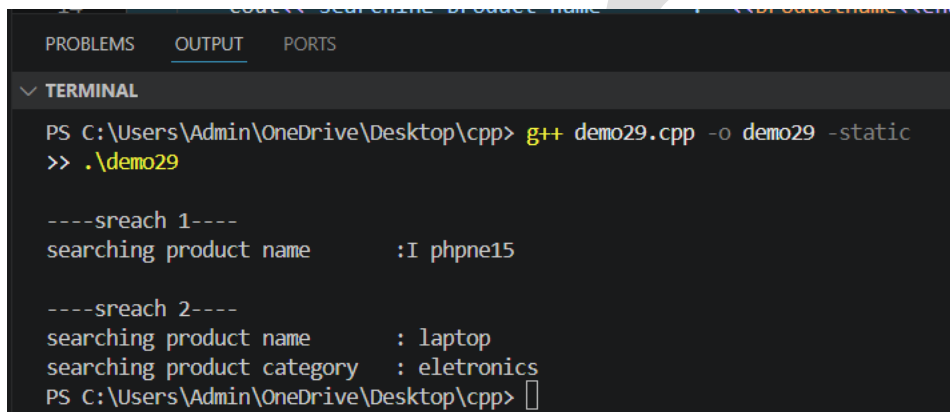
```
cout<<"\n----sreach 2----"<<endl;

search.sreachProduct("laptop","eletronics");

return 0;

}
```

Output:



```
PROBLEMS  OUTPUT  PORTS
▼ TERMINAL
PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo29.cpp -o demo29 -static
>> .\demo29

----sreach 1----
searching product name      :I phne15

----sreach 2----
searching product name      : laptop
searching product category   : eletronics
PS C:\Users\Admin\OneDrive\Desktop\cpp> 
```

- 4) Given this code: `class MusicPlayer { void play(String song) { System.out.println("Playing: " + song); } }` `class SpotifyPlayer extends MusicPlayer { void play(String song) { System.out.println("Streaming on Spotify: " + song); } }` — Create an object of type `MusicPlayer` but assign it a `SpotifyPlayer` instance, then call `play()`. Explain the output.
Hint: This tests runtime polymorphism (overriding) and dynamic method dispatch.

Code-

```
#include<iostream>
#include<string>
using namespace std;

class MusicPlayer{

    public:
    virtual void play(string song){
        cout<<"playing :"<<song<<endl;

    }

};

class SpotifyPlayer:public MusicPlayer{
    public:
    void play(string song)override{
        cout<<"striming on Spotify :"<<song<<endl;
    }
};

int main(){
    MusicPlayer *m1=new SpotifyPlayer();
    m1->play("shape of you");
    delete m1;
    return 0;
}
```

Output:

```
PROBLEMS OUTPUT PORTS
▼ TERMINAL
PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo30.cpp -o demo30 -static
>> .\demo30
striming on Spotify :shape of you
PS C:\Users\Admin\OneDrive\Desktop\cpp> 
```

TOPS